`done_waiting()`, because a thread is declaring that it is done with waiting:

```
void done_waiting()
{
    --count;          <-1
    if(!--spaces)     <-2
    {
        spaces=count.load();     <-3
        ++generation;
    }
}
```

The first thing you do is decrement the `count` ❶ so that the next time `spaces` is reset it reflects the new lower number of waiting threads. Then you need to decrease the number of free `spaces` ❷. If you don't do this, the other threads will be waiting forever, because `spaces` was initialized to the old, larger value. If you're the last thread through on this batch, you need to reset the counter and increase the `generation` ❸, just as you do in `wait()`. The key difference here is that if you're the last thread in the batch, you don't have to wait. You're finished with waiting after all!

You're now ready to write your second implementation of partial sum. At each step, every thread calls `wait()` on the barrier to ensure the threads step through together, and once each thread is done, it calls `done_waiting()` on the barrier to decrement the count. If you use a second buffer alongside the original range, the barrier provides all the synchronization you need. At each step the threads read from either the original range or the buffer and write the new value to the corresponding element of the other. If the threads read from the original range on one step, they read from the buffer on the next, and vice versa. This ensures there are no race conditions between the reads and writes by separate threads. Once a thread has finished looping, it must ensure that the correct final value has been written to the original range. The following listing pulls this all together.

Listing 8.13. A parallel implementation of `partial_sum` by pairwise updates